

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №1 по курсу
«Операционные системы»**

**Тема работы
“Взаимодействие между процессами”**

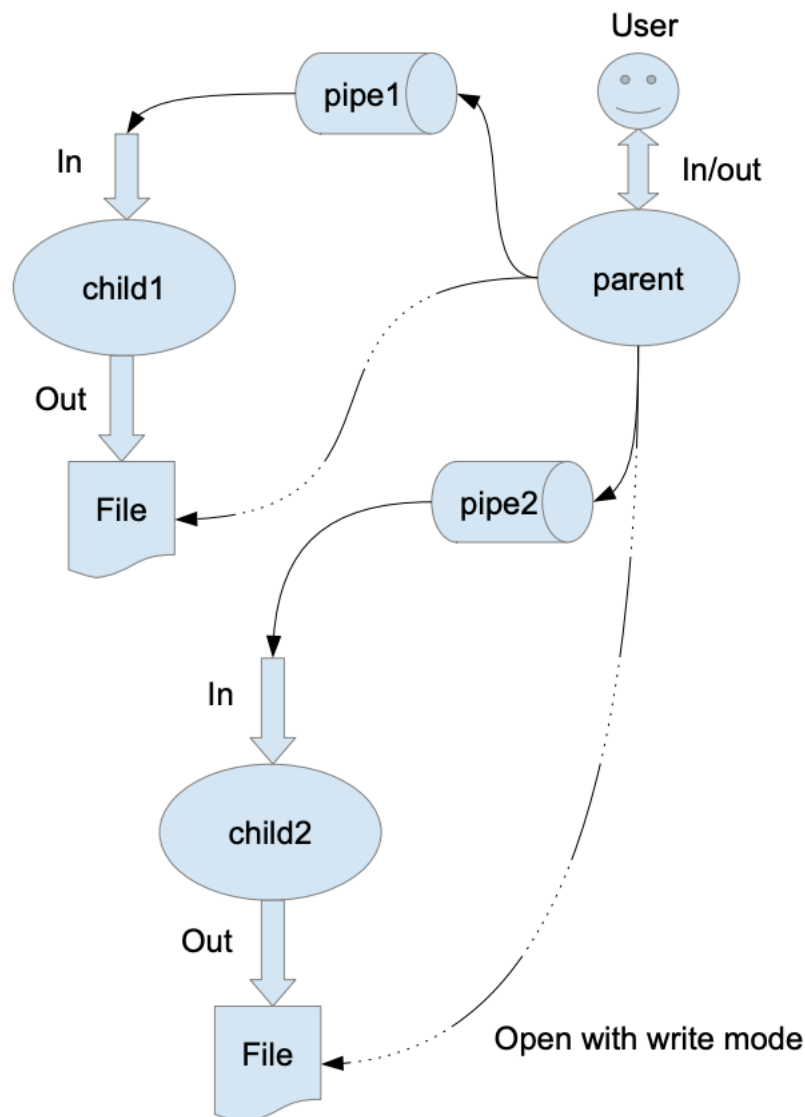
Студент: Салихов Руслан Ринатович
Группа: М8О-203Б-23
Вариант: 19

Преподаватель: Миронов Евгений Сергеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Постановка задачи

Задача: Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работ



Вариант 19) Правило фильтрации: с вероятностью 80% строки отправляются в `pipe1`, иначе в `pipe2`.

Дочерние процессы удаляют все гласные из строк.

Общие сведения

Родительский процесс создает два дочерних процесса. Первой строкой пользователь вводит в консоль родительского процесса имя файла, который будет использоваться для записи в процессе `child1`. Аналогично, второй строкой вводится имя файла для `child2`. Родительский и дочерние процессы представлены разными программами.

Родительский процесс получает от пользователя строки произвольной длины и направляет их либо в `pipe1`, либо в `pipe2` в соответствии с правилом фильтрации: с вероятностью 80% строка отправляется в `pipe1`, иначе в `pipe2`. Оба дочерних процесса принимают строки, обрабатывают их, удаляя все гласные, и выводят результат в стандартный поток вывода.

Вывод

В ходе разработки программы я изучил различные функции, используемые для работы с процессами. `fork` применяется для создания дочерних процессов, `pipe` — для организации межпроцессного взаимодействия, `dup2` — для перенаправления потоков ввода-вывода, а `execv` — для запуска новой программы в контексте дочернего процесса.

Код программы

src/child1.cpp

```
#include <iostream>
#include <string>
#include <unistd.h>
#include "functions.hpp"

int main() {
    processChild();
    return 0;
}
```

src/child2.cpp

```
#include <iostream>
#include <string>
#include <unistd.h>
#include "functions.hpp"

int main() {
    processChild();
    return 0;
}
```

src/functions.cpp

```
#include "functions.hpp"
#include <iostream>
#include <unistd.h>
#include <cstring>
#include <cstdlib>
#include <fcntl.h>
#include <sys/wait.h>

using namespace std;

void CreatePipe(int pipeFd[2]) {
    if (pipe(pipeFd) == -1) {
        perror("Ошибка создания канала");
        exit(EXIT_FAILURE);
    }
}

pid_t CreateChild() {
```

```

pid_t pid = fork();
if (pid == -1) {
    perror("Ошибка создания дочернего процесса");
    exit(EXIT_FAILURE);
}
return pid;
}

void Exec(const char* pathToChild) {
    execl(pathToChild, pathToChild, nullptr);
    perror("Ошибка выполнения дочернего процесса");
    exit(EXIT_FAILURE);
}

void processChild() {
    string input;
    int length;
    while (read(STDIN_FILENO, &length, sizeof(length)) > 0) {
        char* buffer = new char[length + 1];
        read(STDIN_FILENO, buffer, length);
        buffer[length] = '\0';
        input = buffer;
        delete[] buffer;
        // удаляю гласные и вывод результата
        string output = removeVowels(input);
        cout << output << endl;
    }
}

string removeVowels(const string& input) {
    string result;
    for (char ch : input) {
        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result += ch;
        }
    }
    return result;
}

```

src/parent.cpp

```

#include "parent.hpp"
#include "functions.hpp"
#include <iostream>
#include <string>
#include <cstdlib>
#include <ctime>
#include <unistd.h>
#include <sys/wait.h>

```

```

#include <fcntl.h>

using namespace std;

void ParentProcess(const char* pathToChild1, const char* pathToChild2, istream& streamIn)
{
    int child1[2], child2[2];

    // создаю каналы и проверяю успешность их создания
    CreatePipe(child1);

    CreatePipe(child2);
    srand(time(nullptr));

    string file1="out1.txt", file2="out2.txt";
    //cout << "Введите имя файла для child1: ";
    //getline(streamIn, file1);
    //cout << "Введите имя файла для child2: ";
    //getline(streamIn, file2);

    int file1Descr = open(file1.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0777);
    int file2Descr = open(file2.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0777);
    if (file1Descr == -1) {
        cerr << "Ошибка открытия файла для child1: " << file1 << endl;
        exit(1);
    }
    if (file2Descr == -1) {
        cerr << "Ошибка открытия файла для child2: " << file2 << endl;
        exit(1);
    }

    // первый дочерний проц
    pid_t pid1 = CreateChild();
    if (pid1 == -1) {
        cerr << "Ошибка создания первого дочернего процесса." << endl;
        close(file1Descr);
        close(file2Descr);
        exit(1);
    } else if (pid1 == 0) {
        // Код для первого дочернего процесса
        close(child1[WRITE_END]);
        close(child2[READ_END]);
        close(child2[WRITE_END]);
        // STDIN_FILENO — это дескриптор файла для стандартного ввода.
        // dup2 используется для дублирования файлового дескриптора
        //и перенаправления потока ввода или вывода одного дескриптора на другой.
        if (dup2(child1[READ_END], STDIN_FILENO) == -1) {
            perror("Ошибка перенаправления stdin для child1");
            exit(EXIT_FAILURE);
        }
        if (dup2(file1Descr, STDOUT_FILENO) == -1) {

```

```

    perror("Ошибка перенаправления stdout для child1");
    exit(EXIT_FAILURE);
}
close(file1Descr);
Exec(pathToChild1); //заменяет текущий процесс на новый
} else {
    // создаю второй дочерний процесс
    pid_t pid2 = CreateChild();
    if (pid2 == -1) {
        cerr << "Ошибка создания второго дочернего процесса." << endl;
        close(file1Descr);
        close(file2Descr);
        exit(1);
    } else if (pid2 == 0) {
        // Код для второго дочернего процесса
        close(child2[WRITE_END]); //оставляю на запись второй процесс
        close(child1[READ_END]);
        close(child1[WRITE_END]);

        if (dup2(child2[READ_END], STDIN_FILENO) == -1) {
            perror("Ошибка перенаправления stdin для child2");
            exit(EXIT_FAILURE);
        }
        if (dup2(file2Descr, STDOUT_FILENO) == -1) {
            perror("Ошибка перенаправления stdout для child2");
            exit(EXIT_FAILURE);
        }
        close(file2Descr);
        Exec(pathToChild2);
    } else {
        //выполняется только в родительском процессе,
        //после того как он создал оба дочерних процесса.
        close(child1[READ_END]);
        close(child2[READ_END]);

        // Передача строк в один из каналов с вероятностью 80% на pipe1, 20% на pipe2
        string input;
        while (getline(streamIn, input)) {
            int length = input.size();
            int pipe_write_end;

            // с вероятностью 80% отправляем в pipe1, иначе в pipe2
            if (rand() % 10 < 8) {
                pipe_write_end = child1[WRITE_END];
            } else {
                pipe_write_end = child2[WRITE_END];
            }

            // Отправка длины и содержимого строки
            if (write(pipe_write_end, &length, sizeof(length)) == -1) {
                perror("Ошибка записи длины строки в канал");
                break;
            }
        }
    }
}

```

```

    }
    if (write(pipe_write_end, input.c_str(), length) == -1) {
        perror("Ошибка записи строки в канал");
        break;
    }
}

// выполнил закрытие дескрипторов записи после завершения передачи данных
close(child1[WRITE_END]);
close(child2[WRITE_END]);
close(file1Descr);
close(file2Descr);

// ожидает завершения обоих дочерних процессов
waitpid(pid1, nullptr, 0);
waitpid(pid2, nullptr, 0);
}
}
}

```

Пример вывода:

```

user@DESKTOP-Q8E00MG:/mnt/c/Users/user/Desktop/Labs/os/lab1/build$
./my_tests

[=====] Running 2 tests from 1 test suite.

[-----] Global test environment set-up.

[-----] 2 tests from ParentProcessTest

[ RUN      ] ParentProcessTest.ParentTest1

/mnt/c/Users/user/Desktop/Labs/os/lab1/tests/test.cpp:28: Failure
Expected: (pathToChild1) != (nullptr), actual: NULL vs (nullptr)
CHILD1_PATH environment variable is not set

[ PASSED   ] ParentProcessTest.ParentTest1 (0 ms)

[ RUN      ] ParentProcessTest.EmptyString

/mnt/c/Users/user/Desktop/Labs/os/lab1/tests/test.cpp:82: Failure
Expected: (pathToChild1) != (nullptr), actual: NULL vs (nullptr)
CHILD1_PATH environment variable is not set

```



```
[ PASSED ] ParentProcessTest.EmptyString (0 ms)
[-----] 2 tests from ParentProcessTest (0 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (0 ms total)
[ PASSED ] 0 tests.
[ PASSED ] 2 tests, listed below:
[ PASSED ] ParentProcessTest.ParentTest1
[ PASSED ] ParentProcessTest.EmptyString
```